

tf.cast Stay organized with collections

Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/cast

Casts a tensor to a new type.

Function Signatures

```
tf.cast( x, dtype, name=None )
```

Code Examples

```
tf.cast(  
x, dtype, name=None  
)
```

```
tf.cast(  
x, dtype, name=None  
)
```

```
x = tf.constant([1.8, 2.2], dtype=tf.float32)  
tf.cast(x, tf.int32)
```

```
x = tf.constant([1.8, 2.2], dtype=tf.float32)
```

```
tf.cast(x, tf.int32)
```

```
x = tf.constant([1.8, 2.2], dtype=tf.float32)  
tf.dtypes.cast(x, tf.int32)
```

```
x = tf.constant([1.8, 2.2], dtype=tf.float32)
```

Documentation

Casts a tensor to a new type.

View aliases

Main aliases

`tf.dtypes.cast`

See

Migration guide for
more details.

`tf.compat.v1.cast`, `tf.compat.v1.dtypes.cast`

Used in the notebooks

- Multilayer perceptrons for digit recognition with Core APIs
 - Logistic regression for binary classification with Core APIs
 - `tf.data`: Build TensorFlow input pipelines
 - Distributed training with Core APIs and `DTensor`
 - Import a JAX model using JAX2TF
 - Scalable model compression
 - DeepDream
 - Parameter server training with `ParameterServerStrategy`
 - Learned data compression
 - Neural style transfer

The operation casts `x` (in case of `Tensor`) or `x.values` (in case of `SparseTensor` or `IndexedSlices`) to `dtype`.

For example:

Notice `tf.cast` has an alias `tf.dtypes.cast`:

The operation supports data types (for `x` and `dtype`) of `uint8`, `uint16`, `uint32`, `uint64`, `int8`, `int16`, `int32`, `int64`, `float16`, `float32`, `float64`, `complex64`, `complex128`, `bfloat16`.

In case of casting from complex types (`complex64`, `complex128`) to real types, only the real part of `x` is returned. In case of casting from real types to complex types (`complex64`, `complex128`), the imaginary part of the returned value is set to 0. The handling of complex types here matches the behavior of `numpy`.

Note casting `nan` and `inf` values to integral types has undefined behavior.

Note this operation can lead to a loss of precision when converting native Python float and complex variables to `tf.float64` or `tf.complex128` tensors, since the input is first converted to the `float32` data type and then widened. It is recommended to use `tf.convert_to_tensor` instead of `tf.cast` for any non-tensor inputs.

Returns

Raises